

C++



CÓMO PROGRAMAR

NOVENA EDICIÓN

11

PAUL DEITEL
HARVEY DEITEL

Programación orientada a objetos: polimorfismo

12

*El silencio, a menudo de pura
mocencia,
persuade cuando el habla falla.*

—William Shakespeare

*Las proposiciones generales no
deciden casos concretos.*

—Oliver Wendell Holmes

*Un filósofo de imponente
estatura no piensa en un vacío.
Incluso sus ideas más abstractas
son, en cierta medida,
condicionadas
por lo que se conoce o no en el
tiempo en que vive.*

—Alfred North Whitehead

Objetivos

En este capítulo aprenderá a:

- Reconocer que el polimorfismo hace la programación más conveniente y los sistemas más extensibles.
- Distinguir entre clases abstractas y concretas, y a crear clases abstractas.
- Usar la información de tipos en tiempo de ejecución (RTTI).
- Distinguir cómo implementa C++ las funciones virtual y la vinculación dinámica.
- Reconocer cómo es que los destructores virtual aseguran que se ejecuten todos los destructores apropiados en un objeto.

12.1	Introducción	12.6.2	Creación de la clase derivada concreta <code>EmpleadoAsalariado</code>
12.2	Introducción al polimorfismo: videojuego polimórfico	12.6.3	Creación de la clase derivada concreta <code>EmpleadoPorComision</code>
12.3	Relaciones entre los objetos en una jerarquía de herencia	12.6.4	Creación de la clase derivada concreta indirecta <code>EmpleadoBaseMasComision</code>
12.3.1	Invocación de funciones de la clase base desde objetos de una clase derivada	12.6.5	Demostración del procesamiento polimórfico
12.3.2	Cómo orientar los apuntadores de una clase derivada a objetos de la clase base	12.7	(Opcional) Polimorfismo, funciones virtuales y vinculación dinámica "detrás de las cámaras"
12.3.3	Llamadas a funciones miembro de una clase derivada a través de apuntadores de la clase base	12.8	Caso de estudio: sistema de nómina mediante el uso de polimorfismo e información de tipos en tiempo de ejecución con conversión descendente, <code>dynamic_cast</code> , <code>typeid</code> y <code>type_info</code>
12.3.4	Funciones y destructores virtuales	12.9	Conclusión
12.4	Tipos de campos e instrucciones <code>switch</code>		
12.5	Clases abstractas y funciones <code>virtual</code> puras		
12.6	Caso de estudio: sistema de nómina mediante el uso de polimorfismo		
12.6.1	Creación de la clase base abstracta <code>Empleado</code>		

Resumen | Ejercicios de autoevaluación | Respuestas a los ejercicios de autoevaluación | Ejercicios
| Hacer la diferencia

12.1 Introducción

Ahora continuaremos nuestro estudio de la POO al explicar y demostrar el **polimorfismo** con las jerarquías de herencia. El polimorfismo nos permite "programar en *general*" en vez de "programar de manera *específica*". En especial, el polimorfismo nos permite escribir programas que procesen objetos de clases que formen parte de la *misma* jerarquía de clases, como si todos fueran objetos de la clase base de la jerarquía. Como veremos en breve, el polimorfismo trabaja con los *manejadores de apuntadores* de clase base y *manejadores de referencias* de clase base, pero *no* con los manejadores de nombres.

Implementación para extensibilidad

Con el polimorfismo podemos diseñar e implementar sistemas que puedan *extenderse* con facilidad; pueden agregarse nuevas clases con sólo modificar un poco (o nada) las porciones generales del programa, siempre y cuando las nuevas clases sean parte de la jerarquía de herencia que el programa procesa en forma genérica. Las únicas partes de un programa que deben alterarse para dar cabida a las nuevas clases son las que requieren un conocimiento directo de las nuevas clases que el programador va a agregar a la jerarquía. Por ejemplo, si creamos la clase `Tortuga` que hereda de la clase `Animal` (que podría responder a un mensaje `mover` caminando una pulgada), necesitamos escribir sólo la clase `Tortuga` y la parte de la simulación que crea una instancia de un objeto `Tortuga`. Las porciones de la simulación que procesan a cada `Animal` en forma genérica pueden permanecer iguales.

Discusión opcional sobre el polimorfismo "detrás de las cámaras"

Una característica clave de este capítulo es su discusión detallada (opcional) acerca del polimorfismo, las funciones `virtual` y la vinculación dinámica "detrás de las cámaras", la cual utiliza un diagrama detallado para explicar cómo se puede implementar el polimorfismo en C++.

12.2 Introducción al polimorfismo: videojuego polimórfico

Suponga que vamos a diseñar un videojuego que manipule objetos de muchos tipos *distintos*, incluyendo objetos de las clases *Marciano*, *Venusino*, *Plutonio*, *NaveEspacial* y *RayoLaser*. Imagine que cada clase hereda de la clase base común llamada *ObjetoEspacial*, la cual contiene la función miembro *dibujar*. Cada clase derivada implementa a esta función de una manera apropiada para esa clase. Un programa administrador de la pantalla mantiene un contenedor (por ejemplo, un vector) que contiene *apuntadores* *ObjetoEspacial* a objetos de las distintas clases. Para actualizar la pantalla, el administrador de pantalla envía el *mismo* mensaje a cada objeto; a saber, *dibujar*. Cada tipo de objeto responde de manera única. Por ejemplo, un objeto *Marciano* podría dibujarse a sí mismo en color rojo, con el número apropiado de antenas, un objeto *NaveEspacial* podría dibujarse a sí mismo como un platillo volador de color plateado y un objeto *RayoLaser* podría dibujarse a sí mismo como un rayo color rojo brillante a lo largo de la pantalla. El *mismo* mensaje (en este caso, *dibujar*) que se envía a una *variedad* de objetos tiene *muchas formas* de resultados; de aquí que se use el término polimorfismo.

Un administrador de pantalla polimórfico facilita el proceso de agregar nuevas clases a un sistema, con el mínimo de modificaciones a su código. Suponga que deseamos agregar objetos de la clase *Mercuriano* a nuestro videojuego. Para ello, debemos crear una clase *Mercuriano* que herede de *ObjetoEspacial*, pero proporcione su propia definición de la función miembro *dibujar*. Después, cuando aparezcan *apuntadores* a objetos de la clase *Mercuriano* en el contenedor, no será necesario modificar el código para el administrador de pantalla. Éste invocará a la función miembro *dibujar* en *cada* objeto en el contenedor, *sin importar* el tipo del objeto, por lo que los nuevos objetos *Mercuriano* simplemente “se integran en forma automática”. Así, sin modificar el sistema (más que para crear e incluir las mismas clases), los programadores pueden utilizar el polimorfismo para acomodar clases adicionales, incluyendo las que *no se hayan considerado* a la hora de crear el sistema.



Observación de Ingeniería de Software 12.1

El polimorfismo nos permite tratar con las generalidades y dejar que el entorno en tiempo de ejecución se encargue de los detalles específicos. Los programadores pueden ordenar a una variedad de objetos que se comporten en formas apropiadas para ellos, sin necesidad de conocer sus tipos, siempre y cuando éstos pertenezcan a la misma jerarquía de herencia y se utilicen a través de un apuntador o una referencia común de la clase base.



Observación de Ingeniería de Software 12.2

El polimorfismo promueve la extensibilidad: el software escrito para invocar el comportamiento polimórfico se escribe de manera independiente de los tipos de los objetos a los cuales se envían los mensajes. Así, se pueden incorporar en un sistema nuevos tipos de objetos que pueden responder a los mensajes existentes, sin necesidad de modificar el sistema base. Sólo el código cliente que crea instancias de los nuevos objetos debe modificarse para dar cabida a los nuevos tipos.

12.3 Relaciones entre los objetos en una jerarquía de herencia

En la sección 11.3 se creó una jerarquía de clases de empleados, en la cual la clase *EmpleadoBaseMasComision* heredó de la clase *EmpleadoPorComision*. Los ejemplos del capítulo 11 manipularon objetos

EmpleadoPorComision y objetos EmpleadoBaseMasComision mediante el uso de sus nombres para invocar a sus funciones miembro. Ahora examinaremos las relaciones entre las clases en una jerarquía con más detalle. Las siguientes secciones presentan una serie de ejemplos que demuestran cómo se pueden orientar *apuntadores* de la clase base y de la clase derivada a objetos de la clase base y de la clase derivada, y cómo se pueden utilizar esos apuntadores para invocar a funciones miembro que manipulen a esos objetos.

- En la sección 12.3.1 asignaremos la dirección de un objeto de la clase derivada a un apuntador de la clase base, y después mostraremos que al invocar una función a través del apuntador de la clase base se invoca a la *funcionalidad de la clase base* en el objeto de la clase derivada; es decir, *el tipo del manejador determina cuál función se llama*.
- En la sección 12.3.2 asignaremos la dirección de un objeto de la clase base a un apuntador de la clase derivada, lo cual produce un error de compilación. Hablaremos sobre el mensaje de error e investigaremos por qué el compilador *no* permite dicha asignación.
- En la sección 12.3.3 asignaremos la dirección de un objeto de la clase derivada a un apuntador de la clase base, y después examinaremos cómo puede usarse el apuntador de la clase base para invocar sólo la funcionalidad de la clase base; *al tratar de invocar funciones miembro de la clase derivada a través del apuntador de la clase base, se producen errores de compilación*.
- Finalmente, en la sección 12.3.4 demostraremos cómo obtener un comportamiento polimórfico de los apuntadores de la clase base dirigidos a objetos de la clase derivada. Introduciremos las funciones `virtual` y el polimorfismo, al declarar una función de la clase base como `virtual`. Después asignaremos la dirección de un objeto de la clase derivada al apuntador de la clase base y utilizaremos ese apuntador para invocar la funcionalidad de la clase derivada; *precisamente la capacidad que necesitamos para lograr el comportamiento polimórfico*.

Un concepto clave en estos ejemplos es demostrar que con la herencia *public* un objeto de una clase derivada puede tratarse como un objeto de su clase base. Esto permite varias manipulaciones interesantes. Por ejemplo, un programa puede crear un arreglo de apuntadores de la clase base que apunten a objetos de muchos tipos de clases derivadas. A pesar del hecho de que los objetos de las clases derivadas son de *distintos tipos*, el compilador lo permite debido a que cada objeto de una clase derivada *es* un objeto de su clase base. Sin embargo, *no podemos tratar a un objeto de la clase base como un objeto de una de sus clases derivadas*. Por ejemplo, un EmpleadoPorComision no es un EmpleadoBaseMasComision en la jerarquía definida en el capítulo 11; un EmpleadoPorComision *no* tiene un miembro de datos `salarioBase` y *no* tiene las funciones miembro `establecerSalarioBase` y `obtenerSalarioBase`. La relación *es-un* se aplica sólo de una clase derivada a sus clases base directa e indirectas.

12.3.1 Invocación de funciones de la clase base desde objetos de una clase derivada

El ejemplo en la figura 12.1 reutiliza las versiones finales de las clases EmpleadoPorComision y EmpleadoBaseMasComision de la sección 11.3.5. Este ejemplo demuestra tres formas de orientar los apuntadores de la clase base y los apuntadores de la clase derivada a objetos de la clase base y objetos de la clase derivada. Las primeras dos son simples y directas: orientamos un apuntador de la clase base a un objeto de la clase base e invocamos la funcionalidad de la clase base, y orientamos un apuntador de la clase derivada a un objeto de la clase derivada e invocamos la funcionalidad de la clase derivada. Después, demostramos la relación entre las clases derivadas y las clases base (es decir, la relación *es-un* de herencia) al orientar un apuntador de la clase base a un objeto de la clase derivada y mostrar que la funcionalidad de la clase base está evidentemente disponible en el objeto de la clase derivada.

```

1 // Fig. 12.1: fig12_01.cpp
2 // Cómo orientar los apuntadores de la clase base y la clase derivada a los
3 // objetos de la clase base y la clase derivada, respectivamente.
4 #include <iostream>
5 #include <iomanip>
6 #include "EmpleadoPorComision.h"
7 #include "EmpleadoBaseMasComision.h"
8 using namespace std;
9
10 int main()
11 {
12     // crea el objeto de la clase base
13     EmpleadoPorComision empleadoPorComision(
14         "Sue", "Jones", "222-22-2222", 10000, .06);
15
16     // crea un apuntador de la clase base
17     EmpleadoPorComision *empleadoPorComisionPtr = nullptr;
18
19     // crea un objeto de la clase derivada
20     EmpleadoBaseMasComision empleadoBaseMasComision(
21         "Bob", "Lewis", "333-33-3333", 5000, .04, 300);
22
23     // crea un apuntador de la clase derivada
24     EmpleadoBaseMasComision *empleadoBaseMasComisionPtr = nullptr;
25
26     // establece el formato de salida de punto flotante
27     cout << fixed << setprecision( 2 );
28
29     // imprime los objetos empleadoPorComision y empleadoBaseMasComision
30     cout << "Impresión de los objetos de clase base y clase derivada:\n\n";
31     empleadoPorComision.imprimir();
32     // invoca a la función imprimir de la clase base
33     cout << "\n\n";
34     empleadoBaseMasComision.imprimir();
35     // invoca a la función imprimir de la clase derivada
36
37     // orienta el apuntador de la clase base al objeto de la clase base e imprime
38     empleadoPorComisionPtr = &empleadoPorComision; // perfectamente natural
39     cout << "\n\nAl llamar a imprimir con el apuntador de clase base al "
40         << "objeto de clase base se invoca la función imprimir de la clase base:\n\n";
41     empleadoPorComisionPtr->imprimir();
42     // invoca a la función imprimir de la clase base
43
44     // orienta el apuntador de clase derivada al objeto de clase derivada e imprime
45     empleadoBaseMasComisionPtr = &empleadoBaseMasComision; // natural
46     cout << "\n\nAl llamar a imprimir con el apuntador de clase derivada al "
47         << "objeto de clase derivada se invoca a la función imprimir "
48         << "de la clase derivada:\n\n";
49     empleadoBaseMasComisionPtr->imprimir();
50     // invoca a la función imprimir de la clase derivada
51
52     // orienta el apuntador de clase base al objeto de clase derivada e imprime
53     empleadoPorComisionPtr = &empleadoBaseMasComision;
54     cout << "\n\nAl llamar a imprimir con el apuntador de clase base al "
55         << "objeto de clase derivada se invoca a la función imprimir de la "

```

Fig. 12.1 | Asignación de direcciones de objetos de la clase base y la clase derivada a apuntadores de la clase base y la clase derivada (parte 1 de 2).


```

32     << "clase base en ese objeto de la clase derivada:\n\n";
33     empleadoPorComisionPtr->imprimir();
    // invoca a la función imprimir de la clase base
34     cout << endl;
35 } // fin de main

```

Impresión de los objetos de clase base y clase derivada:

```

empleado por comision: Sue Jones
numero de seguro social: 222-22-2222
ventas brutas: 10000.00
tarifa de comision: 0.06

```

```

con salario base empleado por comision: Bob Lewis
numero de seguro social: 333-33-3333
ventas brutas: 5000.00
tarifa de comision: 0.04
salario base: 300.00

```

Al llamar a imprimir con el apuntador de clase base al objeto de clase base se invoca la función imprimir de la clase base:

```

empleado por comision: Sue Jones
numero de seguro social: 222-22-2222
ventas brutas: 10000.00
tarifa de comision: 0.06

```

Al llamar a imprimir con el apuntador de clase derivada al objeto de clase derivada se invoca a la función imprimir de la clase derivada:

```

con salario base empleado por comision: Bob Lewis
numero de seguro social: 333-33-3333
ventas brutas: 5000.00
tarifa de comision: 0.04
salario base: 300.00

```

Al llamar a imprimir con el apuntador de clase base al objeto de clase derivada se invoca a la función imprimir de la clase base en ese objeto de la clase derivada:

```

empleado por comision: Bob Lewis
numero de seguro social: 333-33-3333
ventas brutas: 5000.00
tarifa de comision: 0.04

```

Observe que no se muestra el salario base

Fig. 12.1 | Asignación de direcciones de objetos de la clase base y la clase derivada a apuntadores de la clase base y la clase derivada (parte 2 de 2)

Recuerde que cada objeto `EmpleadoBaseMasComision` es un `EmpleadoPorComision` que también tiene un salario base. La función miembro `ingresos` de la clase `EmpleadoBaseMasComision` (líneas 34 a 37 de la figura 11.15) redefine a la función miembro `ingresos` de la clase `EmpleadoPorComision` (líneas 85 a 88 de la figura 11.14) para incluir el salario base del objeto. La función miembro `imprimir` de la clase `EmpleadoBaseMasComision` (líneas 40 a 48 de la figura 11.15) redefine la versión de la clase `EmpleadoPorComision` (líneas 91 a 98 de la figura 11.14) para mostrar la misma información más el salario base del empleado.

Creación de objetos y visualización de su contenido

En la figura 12.1, en las líneas 13 y 14 se crea un objeto `EmpleadoPorComision` y en la línea 17 se crea un apuntador a un objeto `EmpleadoPorComision`; en las líneas 20 y 21 se crea un objeto `EmpleadoBaseMasComision` y en la línea 24 se crea un apuntador a un objeto `EmpleadoBaseMasComision`. En las líneas 31 y 33 se utiliza el nombre de cada objeto para invocar a su función miembro `imprimir`.

Orientar un apuntador de la clase base a un objeto de la clase base

En la línea 36 se asigna la dirección del objeto `EmpleadoPorComision` de la clase base al apuntador `EmpleadoPorComisionPtr` de la clase base, que la línea 39 utiliza para invocar a la función miembro `imprimir` en ese objeto `EmpleadoPorComision`. Esto invoca a la versión de `imprimir` definida en la clase base `EmpleadoPorComision`.

Orientar un apuntador de la clase derivada a un objeto de la clase derivada

De manera similar, en la línea 42 se asigna la dirección del objeto `EmpleadoBaseMasComision` de la clase derivada al apuntador `EmpleadoBaseMasComisionPtr` de la clase derivada, que la línea 46 utiliza para invocar a la función miembro `imprimir` en ese objeto `EmpleadoBaseMasComision`. Esto invoca a la versión de `imprimir` definida en la clase derivada `EmpleadoBaseMasComision`.

Orientar un apuntador de la clase base a un objeto de la clase derivada

Después, en la línea 49 se asigna la dirección del objeto `EmpleadoBaseMasComision` de la clase derivada al apuntador `EmpleadoPorComisionPtr` de la clase base, que en la línea 53 se utiliza para invocar a la función miembro `imprimir`. Se permite esta “contradicción”, ya que un objeto de una clase derivada *es* un objeto de su clase base. A pesar del hecho de que el apuntador de la clase base `EmpleadoPorComision` apunta a un objeto de la *clase derivada* `EmpleadoBaseMasComision`, se invoca a la función miembro `imprimir` de la *clase base* `EmpleadoPorComision` (en vez de la función `imprimir` de `EmpleadoBaseMasComision`). Los resultados de cada invocación a cada una de las funciones miembro `imprimir` en este programa revelan que *la funcionalidad invocada depende del tipo del manejador (o referencia) que se utiliza para invocar la función, no del tipo del objeto para el que se hace la llamada a la función miembro*. En la sección 12.3.4, cuando presentemos las funciones virtuales, demostraremos que es posible invocar la funcionalidad del tipo del objeto, *en vez de* invocar la funcionalidad del tipo del manejador. Veremos que esto es crucial para implementar el comportamiento polimórfico: el tema clave de este capítulo.

12.3.2 Cómo orientar los apuntadores de una clase derivada a objetos de la clase base

En la sección 12.3.1, asignamos la dirección de un objeto de la clase derivada a un apuntador de la clase base y explicamos que el compilador de C++ permite esta asignación, debido a que un objeto de una clase derivada *es* un objeto de la clase base. Tomamos la metodología opuesta en la figura 12.2, al orientar un apuntador de la clase derivada a un objeto de la clase base. [Nota: este programa reutiliza las versiones finales de las clases `EmpleadoPorComision` y `EmpleadoBaseMasComision` de la sección 11.3.5]. Las líneas 8 y 9 de la figura 12.2 crean un objeto `EmpleadoPorComision`, y en la línea 10 se crea un apuntador `EmpleadoBaseMasComision`. En la línea 14 se trata de asignar la dirección del objeto `EmpleadoPorComision` de la clase base al apuntador `EmpleadoBaseMasComisionPtr` de la clase derivada, pero el compilador genera un error. El compilador evita esta asignación, ya que un `EmpleadoPorComision` *no es* un `EmpleadoBaseMasComision`.

Considere las consecuencias si el compilador permitiera esta asignación. A través de un apuntador `EmpleadoBaseMasComision` podemos invocar *cualquier* función miembro de `EmpleadoBaseMasComision`, incluyendo `establecerSalarioBase`, para el objeto al que apunta el apuntador (es decir, el objeto `EmpleadoPorComision` de la clase base). Sin embargo, el objeto `EmpleadoPorComision` *no* proporciona una función miembro `establecerSalarioBase`, *ni* proporciona un miembro de datos `salarioBase`.

para establecer su valor. Esto podría generar problemas, debido a que la función miembro establecerSalarioBase supondría que hay un miembro de datos salarioBase que se debe establecer en su “ubicación usual” en un objeto EmpleadoBaseMasComision. Esta memoria no pertenece al objeto EmpleadoPorComision, por lo que la función miembro establecerSalarioBase podría sobrescribir otros datos importantes en la memoria, posiblemente datos que pertenezcan a un objeto distinto.

```

1 // Fig. 12.2: fig12_02.cpp
2 // Como orientar un apuntador de clase derivada a un objeto de clase base.
3 #include "EmpleadoPorComision.h"
4 #include "EmpleadoBaseMasComision.h"
5
6 int main()
7 {
8     EmpleadoPorComision empleadoPorComision(
9         "Sue", "Jones", "222-22-2222", 10000, .06 );
10    EmpleadoBaseMasComision *empleadoBaseMasComisionPtr = nullptr;
11
12    // orienta el apuntador de la clase derivada al objeto de la clase base
13    // Error: un EmpleadoPorComision no es un EmpleadoBaseMasComision
14    empleadoBaseMasComisionPtr = &empleadoPorComision;
15 } // fin de main

```

Mensaje de error del compilador Microsoft Visual C++:

```

C:\cpphttp8_ejemplos\cap12\Fig12_02\fig12_02.cpp(14): error C2440: '=' :
cannot convert from 'EmpleadoPorComision *' to 'EmpleadoBaseMasComision *'
Cast from base to derived requires dynamic_cast or static_cast

```

Fig. 12.2 Orientación de un apuntador de la clase derivada a un objeto de la clase base.

12.3.3 Llamadas a funciones miembro de una clase derivada a través de apuntadores de la clase base

Desde un apuntador de la clase base, el compilador nos permite invocar *sólo* a las funciones miembro de la clase base. Por ende, si un apuntador de la clase base se orienta a un objeto de la clase derivada, y se trata de acceder a una *función miembro que sólo pertenezca a la clase derivada*, se producirá un error de compilación.

En la figura 12.3 se muestran las consecuencias de tratar de invocar a una función miembro de la clase derivada desde un apuntador de la clase base. [Nota: estamos reutilizando de nuevo las versiones de las clases EmpleadoPorComision y EmpleadoBaseMasComision de la sección 11.3.5]. En la línea 11 se crea empleadoPorComisionPtr (un apuntador a un objeto EmpleadoPorComision) y en las líneas 12 y 13 se crea un objeto EmpleadoBaseMasComision. En la línea 16 se orienta el apuntador empleadoPorComisionPtr de la clase base al objeto de la clase derivada llamado empleadoBaseMasComision. En la sección 12.3.1 vimos que esto está permitido, ya que un EmpleadoBaseMasComision *es* un EmpleadoPorComision (en el sentido en el que un objeto EmpleadoBaseMasComision contiene toda la funcionalidad de un objeto EmpleadoPorComision). En las líneas 20 a 24 se invocan las funciones miembro de la clase base obtenerPrimerNombre, obtenerApellidoPaterno, obtenerNumeroSeguroSocial, obtenerVentasBrutas y obtenerTarifaComision desde el apuntador de la clase base. Todas estas llamadas son legítimas, ya que EmpleadoBaseMasComision *hereda* estas funciones miembro de EmpleadoPorComision. Sabemos que empleadoPorComisionPtr está orientado a un objeto EmpleadoBaseMasComision, por lo que en las líneas 28 y 29 tratamos de invocar a las funciones miembro de EmpleadoBaseMasComision llamadas obtenerSalarioBase y establecerSalarioBase. El compilador genera errores en ambas llamadas, debido a que *no* se hacen a las funciones miembro de la clase base EmpleadoPorComision. El manejador se

puede utilizar para invocar *sólo* a las funciones que son miembros del tipo de clase asociado de ese manejador. (En este caso, desde un `EmpleadoPorComision *` sólo podemos invocar a las funciones miembro de `EmpleadoPorComision` llamadas `establecerPrimerNombre`, `obtenerPrimerNombre`, `establecerApellidoPaterno`, `obtenerApellidoPaterno`, `establecerNumeroSeguroSocial`, `obtenerNumeroSeguroSocial`, `establecerVentasBrutas`, `obtenerVentasBrutas`, `establecerTarifaComision`, `obtenerTarifaComision`, `ingresos` e `imprimir`).

```

1 // Fig. 12.3: fig12_03.cpp
2 // Intento de invocar a las funciones miembro que sólo son de
3 // la clase derivada a través de un apuntador de la clase base.
4 #include <string>
5 #include "EmpleadoPorComision.h"
6 #include "EmpleadoBaseMasComision.h"
7 using namespace std;
8
9 int main()
10 {
11     EmpleadoPorComision *empleadoPorComisionPtr = nullptr;
12     // apuntador de clase base
13     EmpleadoBaseMasComision empleadoBaseMasComision(
14         "Bob", "Lewis", "333-33-3333", 1000, .04, 300); // clase derivada
15     // orienta el apuntador de la clase base al objeto de la clase derivada
16     // (permitido)
17     empleadoPorComisionPtr = &empleadoBaseMasComision;
18     // invoca a las funciones miembro de la clase base en el objeto de la
19     // clase derivada a través de un apuntador de la clase base (permitido)
20     string primerNombre = empleadoPorComisionPtr->obtenerPrimerNombre();
21     string apellidoPaterno = empleadoPorComisionPtr->obtenerApellidoPaterno();
22     string nss = empleadoPorComisionPtr->obtenerNumeroSeguroSocial();
23     double ventasBrutas = empleadoPorComisionPtr->obtenerVentasBrutas();
24     double tarifaComision = empleadoPorComisionPtr->obtenerTarifaComision();
25
26     // intento de invocar a las funciones miembro que sólo son de la clase derivada
27     // en un objeto de la clase derivada a través de un apuntador de la
28     // clase base (no permitido)
29     double salarioBase = empleadoPorComisionPtr->obtenerSalarioBase();
30     empleadoPorComisionPtr->establecerSalarioBase( 100 );
31 } // fin de main

```

Mensajes de error del compilador GNU C++:

```

fig12_03.cpp:28:47: error: 'class EmpleadoPorComision' has no member named
'obtenerSalarioBase'
fig12_03.cpp:29:27: error: 'class EmpleadoPorComision' has no member named
'establecerSalarioBase'

```

Fig. 12.3 | Intento de invocar a las funciones que sólo son de la clase derivada, a través de un apuntador de la clase base.

Conversión descendente

El compilador permite el acceso a los miembros que sólo son de la clase derivada desde un apuntador de la clase base que esté orientado a un objeto de la clase derivada, si convertimos de manera explícita el apuntador de la clase base a un apuntador de la clase derivada; a esta técnica se le conoce como **conversión descendente**. Como sabe, es posible orientar un apuntador de la clase base a un objeto de la clase derivada,

Sin embargo, como demostramos en la figura 12.3, un apuntador de la clase base se puede usar para invocar *sólo* las funciones declaradas en la clase base. La conversión descendente permite una operación específica de la clase derivada en un objeto de la clase derivada al que apunta un apuntador de la clase base. Después de una conversión descendente, el programa *puede* invocar las funciones de la clase derivada que no están en la clase base. La conversión descendente es una operación potencialmente peligrosa. En la sección 12.8 le mostraremos cómo usar la conversión descendente *en forma segura*.



Observación de Ingeniería de Software 12.3

Si la dirección de un objeto de la clase derivada se ha asignado a un apuntador de una de sus clases base directas o indirectas, es aceptable convertir ese apuntador de la clase base de vuelta a un apuntador del tipo de la clase derivada. De hecho, esto debe hacerse para llamar a las funciones miembro de la clase derivada que no aparecen en la clase base.

12.3.4 Funciones y destructores virtuales

En la sección 12.3.1, orientamos un apuntador de la clase base `EmpleadoPorComision` a un objeto de la clase derivada `EmpleadoBaseMasComision`, y después invocamos a la función miembro `imprimir` a través de ese apuntador. Recuerde que el *tipo del manejador* determina cuál funcionalidad de la clase se va a invocar. En este caso, el apuntador `EmpleadoPorComision` invocó a la función miembro `imprimir` de `EmpleadoPorComision` en el objeto `EmpleadoBaseMasComision`, aun cuando el apuntador estaba orientado a un objeto `EmpleadoBaseMasComision` que tiene su propia función `imprimir` personalizada.



Observación de Ingeniería de Software 12.4

Con las funciones virtual, el tipo del objeto y no el tipo del manejador utilizado para invocar a la función miembro, es el que determina cuál versión de una función virtual se debe invocar.

Por qué son útiles las funciones virtual

Primero vamos a considerar por qué son útiles las funciones virtual. Suponga que un conjunto de clases de figuras como `Círculo`, `Triangulo`, `Rectangulo` y `Cuadrado` se derivan de la clase base `Figura`. Cada una de estas clases podría estar dotada con la habilidad de *dibujarse a sí misma* a través de una función miembro llamada `dibujar`, pero la función para cada figura es bastante distinta. En un programa que dibuja un conjunto de figuras, sería útil poder tratar a todas las figuras en forma genérica como objetos de la clase base `Figura`. Después, para dibujar cualquier figura podríamos simplemente usar un apuntador de la clase base `Figura` para invocar a la función `dibujar`, y dejar que el programa determine en forma *dinámica* (es decir, en tiempo de ejecución) de cuál clase derivada se va a utilizar la función `dibujar`, con base en el tipo del objeto al que apunta el apuntador de la clase base `Figura` en cualquier momento dado. Éste es un *comportamiento polimórfico*.

Declaración de funciones virtual

Para permitir este tipo de comportamiento, declaramos a `dibujar` en la clase base como una **función virtual**, y **sobrescribimos** a `dibujar` en *cada* una de las clases derivadas para dibujar la figura apropiada. Desde una perspectiva de implementación, *sobrescribir* una función no es algo distinto a *redefinirla* (que es la metodología que hemos estado usando hasta ahora). Una función sobrescrita en una clase derivada tiene *la misma firma y el mismo tipo de valor de retorno* (es decir, *prototipo*) que la función que sobrescribe en su clase base. Si no declaramos la función de la clase base como `virtual`, podemos *redefinir* esa función. En contraste, si declaramos la función de la clase base como `virtual`, podemos *sobrescribir* esa función para permitir el *comportamiento polimórfico*. Para declarar una función `virtual`, anteponeamos al prototipo de la función la palabra clave `virtual` en la clase base. Por ejemplo,


```
virtual void dibujar() const;
```

aparecería en la clase base *Figura*. El prototipo anterior declara que la función *dibujar* es una función *virtual* que no recibe argumentos y no devuelve nada. Esta función se declara *const* debido a que, por lo general, una función *dibujar* no realizaría modificaciones al objeto *Figura* en el cual se invoca; las funciones virtuales *no* tienen que ser funciones *const*.



Observación de Ingeniería de Software 12.5

Una vez que una función se declara virtual, permanece virtual en todos los niveles hacia abajo de la jerarquía de herencia desde ese punto, aun si esa función no se declara explícitamente como virtual cuando una clase derivada la sobrescribe.



Buena práctica de programación 12.1

Aun cuando ciertas funciones son implícitamente virtual debido a la declaración que se hace en un nivel superior en la jerarquía de clases, debe declarar explícitamente estas funciones como virtual en cada nivel de la jerarquía de herencia para promover la claridad del programa.



Observación de Ingeniería de Software 12.6

Cuando una clase derivada opta por no sobrescribir una función virtual de su clase base, la clase derivada simplemente hereda la implementación de la función virtual de su clase base.

Invocación de una función virtual a través de un apuntador o referencia de la clase base

Si un programa invoca a una función virtual a través de un apuntador de la clase base a un objeto de la clase derivada (por ejemplo, *figuraPtr->dibujar()*) o una referencia de la clase base a un objeto de la clase derivada (por ejemplo, *figuraRef.dibujar()*), el programa elegirá la función *dibujar* correcta de la clase derivada en forma dinámica (es decir, en tiempo de ejecución) con base en el tipo del objeto, no en el tipo del apuntador o referencia. Al proceso de elegir la función apropiada a llamar en tiempo de ejecución (en vez de hacerlo en tiempo de compilación) se le conoce como **vinculación dinámica** o **vinculación tardía**.

Invocación de una función virtual a través del nombre de un objeto

Cuando una función virtual se llama al referenciar un objeto específico por su nombre y usando el operador punto de selección de miembros (por ejemplo, *objetoCuadrado.dibujar()*), la invocación de la función se resuelve en tiempo de compilación (a esto se le conoce como **vinculación estática**) y la función virtual que se llama es la que se define para (o se hereda por) la clase de ese objeto específico; esto *no* es comportamiento polimórfico. Por ende, la vinculación dinámica con las funciones virtual sólo ocurre a partir de apuntadores (y, como pronto veremos, de referencias).

Funciones virtual en la jerarquía *EmpleadoPorComision*

Ahora veamos cómo las funciones virtual pueden permitir el comportamiento polimórfico en nuestra jerarquía de empleados. Las figuras 12.4 y 12.5 son los encabezados para las clases *EmpleadoPorComision* y *EmpleadoBaseMasComision*, respectivamente. Modificamos estos archivos para declarar las funciones miembro *ingresos* e *imprimir* de cada clase como virtual (líneas 29 y 30 de la figura 12.4, y líneas 19 y 20 de la figura 12.5). Como las funciones *ingresos* e *imprimir* son virtual en la clase *EmpleadoPorComision*, las funciones *ingresos* e *imprimir* de la clase *EmpleadoBaseMasComision* sobrescriben a las de la clase *EmpleadoPorComision*. Además, las funciones *ingresos* e *imprimir* de la clase *EmpleadoBaseMasComision* se declaran como **override**.





Tip para prevenir errores 12.1

Aplique la palabra clave *override* de C++11 a toda función sobrescrita en una clase derivada. Esto obliga al compilador a comprobar si la clase base tiene una función miembro con el mismo nombre y lista de parámetros (es decir, la misma firma). De no ser así, el compilador genera un error.

Ahora, si orientamos un apuntador de la clase base `EmpleadoPorComision` a un objeto de la clase derivada `EmpleadoBaseMasComision`, y el programa utiliza ese apuntador para llamar a la función `ingresos` o `imprimir`, se invocará la función correspondiente del objeto `EmpleadoBaseMasComision`. No hubo modificaciones a las implementaciones de las funciones miembro de las clases `EmpleadoPorComision` y `EmpleadoBaseMasComision`, por lo que reutilizamos las versiones de las figuras 11.14 y 11.15.

```

1 // Fig. 12.4: EmpleadoPorComision.h
2 // Encabezado de la clase EmpleadoPorComision que declara a ingresos e imprimir
  como virtual.
3 #ifndef COMISION_H
4 #define COMISION_H
5
6 #include <string> // clase string estándar de C++
7
8 class EmpleadoPorComision
9 {
10 public:
11     EmpleadoPorComision( const std::string &, const std::string &,
12                          const std::string &, double = 0.0, double = 0.0 );
13
14     void establecerPrimerNombre( const std::string & );
15     // establece el primer nombre
16     std::string obtenerPrimerNombre() const; // devuelve el primer nombre
17
18     void establecerApellidoPaterno( const std::string & );
19     // establece el apellido paterno
20     std::string obtenerApellidoPaterno() const; // devuelve el apellido paterno
21
22     void establecerNumeroSeguroSocial( const std::string & ); // establece el NSS
23     std::string obtenerNumeroSeguroSocial() const; // devuelve el NSS
24
25     void establecerVentasBrutas( double ); // establece el monto de ventas brutas
26     double obtenerVentasBrutas() const; // devuelve el monto de ventas brutas
27
28     void establecerTarifaComision( double ); // establece la tarifa de comisión
29     double obtenerTarifaComision() const; // devuelve la tarifa de comisión
30
31     virtual double ingresos() const; // calcula los ingresos
32     virtual void imprimir() const; // imprime el objeto
33 private:
34     std::string primerNombre;
35     std::string apellidoPaterno;
36     std::string numeroSeguroSocial;
37     double ventasBrutas; // ventas brutas por semana
38     double tarifaComision; // porcentaje de comisión
39 }; // fin de la clase EmpleadoPorComision
40 #endif

```

Fig. 12.4 | Encabezado de la clase `EmpleadoPorComision`, que declara a las funciones `ingresos` e `imprimir` como virtual.

```

1 // Fig. 12.5: EmpleadoBaseMasComision.h
2 // Clase EmpleadoBaseMasComision derivada de la clase
3 // EmpleadoPorComision.
4 #ifndef BASEMAS_H
5 #define BASEMAS_H
6
7 #include <string> // clase string estándar de C++
8 #include "EmpleadoPorComision.h" // declaración de la clase EmpleadoPorComision
9
10 class EmpleadoBaseMasComision : public EmpleadoPorComision
11 {
12 public:
13     EmpleadoBaseMasComision( const std::string &, const std::string &,
14                             const std::string &, double = 0.0, double = 0.0, double = 0.0 );
15
16     void establecerSalarioBase( double ); // establece el salario base
17     double obtenerSalarioBase() const; // devuelve el salario base
18
19     virtual double ingresos() const override; // calcula los ingresos
20     virtual void imprimir() const override; // imprime el objeto
21 private:
22     double salarioBase; // salario base
23 }; // fin de la clase EmpleadoBaseMasComision
24
25 #endif

```

Fig. 12.5 | Encabezado de la clase `EmpleadoBaseMasComision` que declara a las funciones `ingresos` e `imprimir` como virtual.

Modificamos la figura 12.1 para crear el programa de la figura 12.6. En las líneas 40 a 51 de la figura 12.6 se demuestra otra vez que un apuntador `EmpleadoPorComision` orientado a un objeto `EmpleadoPorComision` puede utilizarse para invocar la funcionalidad de `EmpleadoPorComision`, y que un apuntador `EmpleadoBaseMasComision` orientado a un objeto `EmpleadoBaseMasComision` puede utilizarse para invocar la funcionalidad de `EmpleadoBaseMasComision`. En la línea 54 se orienta el apuntador `empleadoPorComisionPtr` de la clase base al objeto `empleadoBaseMasComision` de la clase derivada. Observe que cuando en la línea 61 se invoca a la función miembro `imprimir` desde el apuntador de la clase base, se invoca a la función miembro `imprimir` de la clase derivada `EmpleadoBaseMasComision`, por lo que en la línea 61 se imprime un texto distinto al de la línea 53 en la figura 12.1 (cuando la función miembro `imprimir` *no* se declaró virtual). Podemos ver que, al declarar una función miembro virtual, el programa determina en forma dinámica cuál función debe invocar *con base en el tipo de objeto al que apunta el manejador, en vez de basarse en el tipo del manejador*. Observe de nuevo que cuando `empleadoPorComisionPtr` apunta a un objeto `EmpleadoPorComision`, se invoca a la función `imprimir` de la clase `EmpleadoPorComision` (figura 12.6, línea 40) y cuando `empleadoPorComisionPtr` apunta a un objeto `EmpleadoBaseMasComision`, se invoca a la función `imprimir` de `EmpleadoBaseMasComision` (línea 61). Por ende, el mismo mensaje (`imprimir`, en este caso) que se envía (desde un apuntador de la clase base) a una variedad de objetos relacionados por la herencia a esa clase base, toma muchas formas; éste es el comportamiento polimórfico.


```

1 // Fig. 12.6: fig12_06.cpp
2 // Introducción al polimorfismo, las funciones virtuales y la vinculación
  postergada.
3 #include <iostream>
4 #include <iomanip>
5 #include "EmpleadoPorComision.h"
6 #include "EmpleadoBaseMasComision.h"
7 using namespace std;
8
9 int main()
10 {
11     // crea un objeto de la clase base
12     EmpleadoPorComision empleadoPorComision(
13         "Sam", "Jones", "222-22-2222", 10000, .06 );
14
15     // crea un apuntador de la clase base
16     EmpleadoPorComision *empleadoPorComisionPtr = nullptr;
17
18     // crea un objeto de la clase derivada
19     EmpleadoBaseMasComision empleadoBaseMasComision(
20         "Rob", "Lewis", "333-33-3333", 5000, .04, 100 );
21
22     // crea un apuntador de la clase derivada
23     EmpleadoBaseMasComision *empleadoBaseMasComisionPtr = nullptr;
24
25     // establece el formato de salida de punto flotante
26     cout << fixed << setprecision( 2 );
27
28     // imprime los objetos usando la vinculación estática
29     cout << "Invocando a la función imprimir en objetos de la clase base "
30         << "\ny la clase derivada con vinculación estática\n\n";
31     empleadoPorComision.imprimir(); // vinculación estática
32     cout << "\n\n";
33     empleadoBaseMasComision.imprimir(); // vinculación estática
34
35     // imprime los objetos usando vinculación dinámica
36     cout << "\n\n\nInvocando a la función imprimir en objetos de la clase base "
37         << "\ny la clase derivada con vinculación dinámica";
38
39     // orienta el apuntador de la clase base al objeto de la clase base e imprime
40     empleadoPorComisionPtr = &empleadoPorComision;
41     cout << "\n\nAl llamar a la función virtual imprimir con un apuntador "
42         << "\nde la clase base a un objeto de la clase base se invoca a la "
43         << "función imprimir de la clase base:\n\n";
44     empleadoPorComisionPtr->imprimir();
45     // invoca a la función imprimir de la clase base
46
47     // orienta un apuntador de la clase derivada al objeto de la
48     // clase derivada e imprime
49     empleadoBaseMasComisionPtr = &empleadoBaseMasComision;
50     cout << "\n\nAl llamar a la función virtual imprimir con un apuntador "
51         << "\nde la clase derivada a un objeto de la clase derivada se invoca a "
52         << "la función imprimir de la clase derivada:\n\n";
53     empleadoBaseMasComisionPtr->imprimir();
54     // invoca a la función imprimir de la clase derivada

```

Fig. 12.6 Demostración del polimorfismo al invocar una función virtual de la clase derivada a través de un apuntador de la clase base a un objeto de la clase derivada (parte 1 de 2).

```

32
33 // orienta un apuntador de la clase base a un objeto de la clase derivada e
    imprime
34 empleadoPorComisionPtr = &empleadoBaseMasComision;
35 cout << "\nAl llamar a la función virtual imprimir con un apuntador de la
    clase base"
36     << "\na un objeto de la clase derivada se invoca a la función "
37     << "imprimir de la clase derivada:\n\n";
38
39 // polimorfismo; invoca a la función imprimir de EmpleadoBaseMasComision;
40 // apuntador de la clase base a un objeto de la clase derivada
41 empleadoPorComisionPtr->imprimir();
42 cout << endl;
43 } // fin de main

```

Invocando a la función imprimir en objetos de la clase base y la clase derivada con vinculación estática

empleado por comision: Sue Jones
 numero de seguro social: 222-22-2222
 ventas brutas: 10000.00
 tarifa de comision: 0.06

con salario base empleado por comision: Bob Lewis
 numero de seguro social: 333-33-3333
 ventas brutas: 5000.00
 tarifa de comision: 0.04
 salario base: 300.00

Invocando a la función imprimir en objetos de la clase base y la clase derivada con vinculación dinámica

Al llamar a la función virtual imprimir con un apuntador de la clase base a un objeto de la clase base se invoca a la función imprimir de la clase base:

empleado por comision: Sue Jones
 numero de seguro social: 222-22-2222
 ventas brutas: 10000.00
 tarifa de comision: 0.06

Al llamar a la función virtual imprimir con un apuntador de la clase derivada a un objeto de la clase derivada se invoca a la función imprimir de la clase derivada:

con salario base empleado por comision: Bob Lewis
 numero de seguro social: 333-33-3333
 ventas brutas: 5000.00
 tarifa de comision: 0.04
 salario base: 300.00

Al llamar a la función virtual imprimir con un apuntador de la clase base a un objeto de la clase derivada se invoca a la función imprimir de la clase derivada:

con salario base empleado por comision: Bob Lewis
 numero de seguro social: 333-33-3333
 ventas brutas: 5000.00
 tarifa de comision: 0.04
 salario base: 300.00 ————— Observe que no se muestra el salario base

Fig. 12.6 | Demostración del polimorfismo al invocar una función virtual de la clase derivada a través de un apuntador de la clase base a un objeto de la clase derivada (parte 2 de 2).

Destructores virtual

Puede ocurrir un problema al utilizar polimorfismo para procesar los objetos de una jerarquía de clases que se asignan en forma dinámica. Hasta ahora hemos visto los destructores que no se declaran con la palabra clave `virtual`. Si se destruye un objeto de la clase derivada con un destructor no `virtual` mediante la aplicación del operador `delete` a un *apuntador de la clase base* al objeto, el estándar de C++ especifica que el comportamiento es *indefinido*.

La solución simple para este problema es crear un **destructor virtual** en la clase base. Si el destructor de una clase se declara `virtual`, los destructores de todas las clases derivadas serán *también virtual* y *sobrescribirán* al destructor de la clase base. Por ejemplo, en la definición de la clase `EmpleadoPorComision`, podemos definir el destructor `virtual` de la siguiente manera:

```
virtual ~EmpleadoPorComision() { }
```

Ahora, si un objeto en la jerarquía se destruye explícitamente al aplicar el operador `delete` a un *apuntador de la clase base*, se hace una llamada al destructor para la *clase apropiada* con base en el objeto al que apunta el apuntador de la clase base. Recuerde, cuando se destruye un objeto de la clase derivada, la parte del objeto de la clase derivada correspondiente a la clase base también se destruye, por lo que es importante que se ejecuten *ambos* destructores de la clase derivada y la clase base. El destructor de la clase base se ejecuta de manera automática, después del destructor de la clase derivada. De aquí en adelante, incluiremos un destructor virtual en *cada* clase que contenga funciones `virtual`.



Tip para prevenir errores 12.2

Si una clase tiene funciones `virtual`, siempre debemos proporcionar un destructor `virtual` aunque no se requiera uno para la clase. Esto asegura que se invoque a un destructor personalizado de la clase derivada (si hay uno) cuando se elimine un objeto de la clase derivada mediante un apuntador de la clase base.



Error común de programación 12.1

Los constructores no pueden ser `virtual`. Declarar un constructor como `virtual` es un error de compilación.



C++11: Funciones miembro y clases `final`

Antes de C++11, una clase derivada podía sobrescribir *cualquiera* de las funciones `virtual` de su clase base. En C++11, una función `virtual` de la clase base que se declara **final** en su prototipo, como en

```
virtual unaFunción( parámetros ) final;
```

no puede sobrescribirse en ninguna clase derivada; esto garantiza que la definición de la función miembro `final` de la clase base sea utilizada por todos los objetos de la clase base y por todos los objetos de las clases derivadas directas e indirectas de la clase base. De manera similar, antes de C++11 era posible usar *cualquier* clase existente como clase base en una jerarquía. A partir de C++11, podemos declarar una clase como `final` para evitar que se utilice como una clase base, como en el siguiente ejemplo:

```
class MiClase final // esta clase no puede ser una clase base
{
    // cuerpo de la clase
};
```

Si intentamos sobrescribir una función miembro `final` o heredar de una clase base `final`, se produce un error de compilación.

12.4 Tipos de campos e instrucciones switch

Una manera de determinar el tipo de un objeto es utilizar una instrucción `switch` para comprobar el valor de un campo en el objeto. Esto nos permite diferenciar entre los tipos de los objetos, y después invocar una acción apropiada para un objeto específico. Por ejemplo, en una jerarquía de figuras en las que cada objeto figura tiene un atributo `tipoFigura`, una instrucción `switch` podría comprobar el `tipoFigura` del objeto para determinar cuál función `imprimir` debe llamar.

El uso de la lógica de `switch` expone a los programas a una variedad de problemas potenciales. Por ejemplo, el programador podría olvidar incluir una prueba de tipos cuando sea obligatorio hacerla, o podría olvidar evaluar todos los casos posibles en una instrucción `switch`. Al modificar un sistema basado en `switch` agregando nuevos tipos, el programador podría olvidar insertar los nuevos casos en *todas* las instrucciones `switch` relevantes. Cada adición o eliminación de una clase requiere la modificación de todas las instrucciones `switch` en el sistema; rastrear estas instrucciones puede ser un proceso que consuma mucho tiempo y esté propenso a errores.



Observación de Ingeniería de Software 12.7

La programación polimórfica puede eliminar la necesidad de la lógica de switch. Al utilizar el mecanismo polimórfico para realizar la lógica equivalente, los programadores pueden evitar el tipo de errores que se asocian generalmente con la lógica de switch.



Observación de Ingeniería de Software 12.8

Una consecuencia interesante de utilizar el polimorfismo es que los programas toman una apariencia simplificada. Contienen menos lógica de bifurcación y un código secuencial más simple.

12.5 Clases abstractas y funciones virtual puras

Cuando pensamos en una clase como un tipo, suponemos que los programas crearán objetos de ese tipo. Sin embargo, hay casos en los que es útil definir las *clases de las cuales nunca se tendrá la intención de instanciar objetos*. Dichas clases se conocen como **clases abstractas**. Como estas clases se utilizan comúnmente como clases base en las jerarquías de herencia, nos referimos a ellas como **clases base abstractas**. Estas clases no se pueden utilizar para instanciar objetos ya que, como veremos pronto, las clases abstractas están *incompletas*; las clases derivadas deben definir las "piezas faltantes" antes de poder instanciar objetos de estas clases. En la sección 12.6 crearemos programas con clases abstractas.

Una clase abstracta es una clase base a partir de la cual otras clases puedan heredar. Las clases que se pueden utilizar para instanciar objetos se conocen como **clases concretas**. Dichas clases definen o heredan implementaciones para *todas* las funciones miembro que declaran. Podríamos tener una clase *abstracta* llamada `FiguraBidimensional` y derivar de ella clases *concretas* tales como `Cuadrado`, `Círculo` y `Triángulo`. Podríamos tener también una clase base *abstracta* llamada `FiguraTridimensional` y derivar de ella clases *concretas* tales como `Cubo`, `Esfera` y `Cilindro`. Las clases base abstractas son *demasiado genéricas* como para definir objetos reales; necesitamos *ser más específicos* antes de poder pensar en instanciar objetos. Por ejemplo, si alguien nos dice que "dibujemos la figura bidimensional", ¿qué figura dibujaríamos? Las clases concretas proporcionan los detalles *específicos* que hacen que sea razonable instanciar objetos.

Una jerarquía de herencia *no* necesita contener clases abstractas, pero muchos sistemas orientados a objetos tienen jerarquías de clases encabezadas por clases base abstractas. En ciertos casos, las clases abstractas constituyen unos cuantos niveles superiores de la jerarquía. Un buen ejemplo de ello es la jerarquía de figuras en la figura 11.3, la cual empieza con la clase base abstracta `Figura`. En el siguiente

nivel de la jerarquía tenemos dos clases base abstractas más; a saber, `FiguraBidimensional` y `FiguraTridimensional`. El siguiente nivel de la jerarquía define clases *concretas* para las figuras bidimensionales (a saber, `Círculo`, `Cuadrado` y `Triangulo`) y para las figuras tridimensionales (a saber, `Esfera`, `Cubo` y `Tetraedro`).

Funciones virtuales puras

Para hacer una clase abstracta, se declara una o más de sus funciones `virtual` como “puras”. Una **función virtual pura** se especifica colocando “= 0” en su declaración, como en

```
virtual void dibujar() const = 0; // función virtual pura
```

El “= 0” se conoce como un **especificador puro**. Por lo general, las funciones `virtual` puras *no* proporcionan implementaciones, aunque pueden hacerlo. Cada clase derivada *concreta debe sobrescribir a todas* las funciones `virtual` puras de la clase base con implementaciones concretas de esas funciones; de lo contrario, la clase derivada también es abstracta. La diferencia entre una función `virtual` y una función `virtual` pura es que una función `virtual` *tiene* una implementación y proporciona a la clase derivada la *opción* de sobrescribir la función; en contraste, una función `virtual` pura *no* proporciona una implementación y *requiere* que la clase derivada sobrescriba la función para que esa clase derivada sea concreta; en caso contrario, la clase derivada permanece *abstracta*.

Las funciones `virtual` puras se utilizan cuando *no* tiene sentido para la clase base tener una implementación de una función, pero es conveniente que todas las clases derivadas concretas implementen la función. Regresando a nuestro ejemplo anterior de los objetos espaciales, no tiene sentido para la clase base `ObjetoEspacial` tener una implementación para la función `dibujar` (ya que no hay forma de dibujar un objeto espacial genérico sin tener más información acerca del tipo de objeto espacial que se va a dibujar). Un ejemplo de una función que se definiría como `virtual` (y no como `virtual` pura) sería una que devuelve un nombre para el objeto. Podemos nombrar a un `ObjetoEspacial` genérico (por ejemplo, como “objeto espacial”), por lo que se puede proporcionar una implementación predeterminada para esta función, y la función no necesita ser `virtual` pura. Sin embargo, la función se sigue declarando como `virtual`, ya que se espera que las clases derivadas sobrescriban esta función para proporcionar nombres *más específicos* para los objetos de la clase derivada.



Observación de Ingeniería de Software 12.9

Una clase abstracta define una interfaz pública común para las diversas clases en una jerarquía de clases. Una clase abstracta contiene una o más funciones `virtual` puras que las clases derivadas concretas deben sobrescribir.



Error común de programación 12.2

Si no se sobrescribe una función `virtual` pura en una clase derivada, esa clase se hace abstracta. Tratar de instanciar un objeto de una clase abstracta produce un error de compilación.



Observación de Ingeniería de Software 12.10

Una clase abstracta tiene por lo menos una función `virtual` pura. Una clase abstracta también puede tener miembros de datos y funciones concretas (incluyendo los constructores y destructores), que están sujetos a las reglas normales de la herencia por las clases derivadas.

Aunque *no* podemos instanciar objetos de una clase base abstracta, *sí* podemos usar la clase base abstracta para declarar *apuntadores* y *referencias* que puedan referirse a objetos de cualquier clase *concreta*.

derivada de la clase abstracta. Por lo general, los programas utilizan dichos apuntadores y referencias para manipular los objetos de la clase derivada mediante el polimorfismo.

Controladores de dispositivos y polimorfismo

El polimorfismo es particularmente efectivo para implementar *sistemas de software en niveles*. Por ejemplo, en los sistemas operativos cada tipo de dispositivo físico podría operar en forma muy distinta de los otros. Aun así, los comandos para *leer* o *escribir* datos desde y hacia los dispositivos podrían tener cierta uniformidad. El mensaje *escribir* que se envía a un objeto *controlador de dispositivo* necesita interpretarse de manera específica en el contexto de ese controlador de dispositivo, y en la forma en que ese controlador de dispositivo manipula dispositivos de un tipo específico. Sin embargo, la llamada *escribir* en sí en realidad no es distinta de la *escritura* a cualquier otro dispositivo en el sistema; se coloca cierto número de bytes de memoria en ese dispositivo. Un sistema operativo orientado a objetos podría utilizar una clase base abstracta para proporcionar una interfaz apropiada para todos los controladores de dispositivos. Después, a través de la herencia de esa clase base abstracta, se forman clases derivadas que operen todas de manera similar. Las herramientas (es decir, las funciones `public`) ofrecidas por los controladores de dispositivos se proporcionan como funciones `virtual` puras en la clase base abstracta. Las implementaciones de esas funciones `virtual` puras se proporcionan en las clases derivadas que corresponden a los tipos específicos de controladores de dispositivos. Esta arquitectura también permite *agregar* nuevos dispositivos a un sistema con facilidad. El usuario simplemente puede conectar el dispositivo e instalar el controlador de su nuevo dispositivo. El sistema operativo “habla” con este nuevo dispositivo a través de su controlador de dispositivos, que tiene las mismas funciones miembro `public` que todos los demás controladores de dispositivos; aquellas definidas en la clase base abstracta del controlador de dispositivos.

12.6 Caso de estudio: sistema de nómina mediante el uso de polimorfismo

En esta sección volvemos a examinar la jerarquía `EmpleadoPorComision-EmpleadoBaseMasComision` que exploramos a lo largo de la sección 11.3. En este ejemplo utilizamos una clase abstracta y polimorfismo para realizar cálculos de nómina con base en el tipo de empleado. Creamos una jerarquía de empleados mejorada para resolver el siguiente problema:

Una empresa paga semanalmente a sus empleados. Los empleados son de tres tipos: los empleados asalariados reciben un salario semanal fijo, sin importar el número de horas trabajadas; los empleados por comisión reciben un porcentaje de sus ventas y los empleados por comisión con salario base reciben un salario base, más un porcentaje de sus ventas. Para el periodo de pago actual, la empresa ha decidido recompensar a los empleados con salario base más comisión, agregando un 10 por ciento a su salario base. La empresa desea implementar un programa en C++ que realice sus cálculos de nómina por medio del polimorfismo.

Utilizamos la clase abstracta `Empleado` para representar el concepto general de un empleado. Las clases que se derivan directamente de `Empleado` son: `EmpleadoAsalariado` y `EmpleadoPorComision`. La clase `EmpleadoBaseMasComision` (que se deriva de `EmpleadoPorComision`) representa el último tipo de empleado. El diagrama de clases de UML de la figura 12.7 muestra la jerarquía de herencia para nuestra aplicación de nómina de empleados polimórfica. El nombre de la clase abstracta `Empleado` está en *cursiva*, según la convención del UML.

La clase base abstracta `Empleado` declara la “interfaz” para la jerarquía; es decir, el conjunto de funciones miembro que un programa puede invocar en todos los objetos `Empleado`. Cada empleado, sin importar la forma en que se calculen sus ingresos, tiene un primer nombre, un apellido paterno y un número de seguro social, por lo que aparecen los datos miembro `private` `primerNombre`, `apellidoPaterno` y `numeroSeguroSocial` en la clase base abstracta `Empleado`.



Fig. 12.7 | Diagrama de clases de UML de la jerarquía Empleado.



Observación de Ingeniería de Software 12.11

Una clase derivada puede heredar la interfaz y/o implementación de una clase base. Las jerarquías diseñadas para la **herencia de implementación** tienden a tener su funcionalidad en un nivel alto en la jerarquía; cada nueva clase derivada hereda una o más funciones miembro que se definieron en una clase base, y la clase derivada utiliza las definiciones de la clase base. Las jerarquías diseñadas para la **herencia de interfaz** tienden a tener su funcionalidad en un nivel bajo en la jerarquía; una clase base especifica una o más funciones que deben definirse para cada clase en la jerarquía (es decir, tienen el mismo prototipo), pero las clases derivadas individuales proporcionan sus propias implementaciones de la(s) función(es).

En las siguientes secciones se implementa la jerarquía de la clase Empleado. Las primeras cinco secciones implementan cada una de las clases abstractas o concretas. La última sección implementa un programa de prueba que crea objetos de todas estas clases y procesa los objetos mediante el polimorfismo.

12.6.1 Creación de la clase base abstracta Empleado

La clase Empleado (figuras 12.9 y 12.10, que veremos con detalle en breve) proporciona las funciones ingresos e imprimir, además de varias funciones establecer y obtener que manipulan los miembros de datos Empleado. Una función ingresos se aplica evidentemente en forma genérica a todos los empleados, pero cada cálculo de los ingresos depende de la clase del empleado. Por lo tanto, declaramos ingresos como virtual pura en la clase base Empleado debido a que una implementación predeterminada no tiene sentido para esa función; no hay suficiente información para determinar qué cantidad debe devolver ingresos. Cada clase derivada sobrescribe a ingresos con una implementación apropiada. Para calcular los ingresos de un empleado, el programa asigna la dirección de un objeto empleado a un apuntador de la clase base Empleado, y después invoca a la función ingresos en ese objeto. Mantenemos un vector de apuntadores Empleado, cada uno de los cuales apunta a un objeto Empleado. Desde luego, no puede haber objetos Empleado debido a que es una clase base abstracta; sin embargo, debido a la herencia todos los objetos de todas las clases derivadas de Empleado pueden considerarse como objetos Empleado. El programa itera a través del vector y llama a la función ingresos para cada objeto Empleado. C++ procesa estas llamadas a funciones en forma polimórfica. Al incluir a ingresos como una función virtual pura en Empleado, se obliga a todas las clases derivadas directas de Empleado que deseen ser una clase concreta a sobrescribir ingresos.

La función imprimir en la clase Empleado muestra el primer nombre, apellido paterno y número de seguro social del empleado. Como veremos, cada clase derivada de Empleado sobrescribe a la función

imprimir para mostrar el tipo del empleado (por ejemplo, "empleado asalariado"), seguido del resto de la información del empleado. La función `imprimir` en las clases derivadas podría llamar también a `ingresos`, aun cuando ésta sea una función virtual pura en la clase base `Empleado`.

El diagrama de la figura 12.8 muestra a cada una de las cuatro clases en la jerarquía hacia abajo en el lado izquierdo, y a las funciones `ingresos` e `imprimir` a lo largo de la parte superior. Para cada clase, el diagrama muestra los resultados deseados de cada función. El texto en cursiva representa en dónde se utilizan los valores de un objeto específico en las funciones `ingresos` e `imprimir`. La clase `Empleado` especifica "= 0" para la función `ingresos`, para indicar que es una función virtual pura y, por ende, no tiene implementación. Cada clase derivada sobrescribe a esta función para proporcionar una implementación apropiada. No listamos las funciones `establecer` y `obtener` de la clase base `Empleado` debido a que no se sobrescriben en ninguna de las clases derivadas; cada una de estas funciones es heredada y utilizada "así como está" por cada una de las clases derivadas.

	ingresos	imprimir
Empleado	= 0	<i>primerNombre</i> <i>apellidoPaterno</i> numero de seguro social: NSS
Empleado-Asalariado	<i>salarioSemanal</i>	empleado asalariado: <i>primerNombre</i> <i>apellidoPaterno</i> numero de seguro social: NSS salario semanal: <i>salarioSemanal</i>
Empleado- PorComision	<i>tarifaComision</i> * <i>ventasBrutas</i>	empleado por comision: <i>primerNombre</i> <i>apellidoPaterno</i> numero de seguro social: NSS ventas brutas: <i>ventasBrutas</i> ; tarifa de comision: <i>tarifaComision</i>
Empleado- Base- MasComision	(<i>tarifaComision</i> * <i>ventasBrutas</i>) + <i>salarioBase</i>	empleado por comision con salario base: <i>primerNombre</i> <i>apellidoPaterno</i> numero de seguro social: NSS ventas brutas: <i>ventasBrutas</i> ; tarifa de comision: <i>tarifaComision</i> ; salario base: <i>salarioBase</i>

Fig. 12.8 | Interfaz polimórfica para las clases de la jerarquía `Empleado`.

Encabezado de la clase `Empleado`

Vamos a considerar el encabezado de la clase `Empleado` (figura 12.9). Las funciones miembro `public` incluyen un constructor que recibe el primer nombre, apellido paterno y número de seguro social como argumentos (línea 11 y 12); un destructor virtual (línea 13); funciones `establecer` que establecen el primer nombre, apellido paterno y número de seguro social (líneas 15, 18 y 21, respectivamente); funciones `obtener` que devuelven el primer nombre, apellido paterno y número de seguro social (líneas 16, 19 y 22, respectivamente); la función virtual pura `ingresos` (línea 25) y la función virtual `imprimir` (línea 26).

```

1 // Fig. 12.9: Empleado.h
2 // Clase base abstracta Empleado.
3 #ifndef EMPLEADO_H

```

Fig. 12.9 | La clase base abstracta `Empleado` (parte 1 de 2).


```

4  #define EMPLEADO_H
5
6  #include <string> // clase string estándar de C++
7
8  class Empleado
9  {
10 public:
11     Empleado( const std::string &, const std::string &,
12              const std::string & );
13     virtual ~Empleado() {} // destructor virtual
14
15     void establecerPrimerNombre( const std::string & ); // establece el primer nombre
16     std::string obtenerPrimerNombre() const; // devuelve el primer nombre
17
18     void establecerApellidoPaterno( const std::string & );
19     // establece el apellido paterno
20     std::string obtenerApellidoPaterno() const; // devuelve el apellido paterno
21
22     void establecerNumeroSeguroSocial( const std::string & ); // establece el NSS
23     std::string obtenerNumeroSeguroSocial() const; // devuelve el NSS
24
25     // la función virtual pura hace de Empleado una clase base abstracta
26     virtual double ingresos() const = 0; // virtual pura
27     virtual void imprimir() const; // virtual
28 private:
29     std::string primerNombre;
30     std::string apellidoPaterno;
31     std::string numeroSeguroSocial;
32 }; // fin de la clase Empleado
33 #endif // EMPLEADO_H

```

Fig. 12.9 | La clase base abstracta `Empleado` (parte 2 de 2).

Recuerde que declaramos a `ingresos` como una función virtual pura, debido a que primero debemos conocer el tipo de `Empleado` *específico* para determinar los cálculos apropiados de ingresos. Al declarar esta función como virtual pura se indica que cada clase derivada concreta *debe* proporcionar una implementación para ingresos, y que un programa puede usar apuntadores de la clase base `Empleado` para invocar a la función ingresos de manera *polimórfica* para *cualquier* tipo de `Empleado`.

Definiciones de las funciones miembro de la clase `Empleado`

La figura 12.10 contiene las implementaciones de las funciones miembro para la clase `Empleado`. No se proporciona una implementación para la función virtual `ingresos`. El constructor de `Empleado` (líneas 9 a 14) no valida el número de seguro social. Por lo general, se debe proporcionar dicha validación.

```

1  // Fig. 12.10: Empleado.cpp
2  // Definiciones de las funciones miembro de la clase base abstracta Empleado.
3  // Nota: no se proporcionan definiciones para las funciones virtuales puras.
4  #include <iostream>
5  #include "Empleado.h" // definición de la clase Empleado
6  using namespace std;

```

Fig. 12.10 | Archivo de implementación de la clase `Empleado` (parte 1 de 2).

```

7
8 // constructor
9 Empleado::Empleado( const string &nombre, const string &apellido,
10 const string &nss )
11 : primerNombre( nombre ), apellidoPaterno( apellido ),
12   numeroSeguroSocial( nss )
13 {
14   // cuerpo vacío
15 } // fin del constructor de Empleado
16
17 // establece el primer nombre
18 void Empleado::establecerPrimerNombre( const string &nombre )
19 {
20   primerNombre = nombre;
21 } // fin de la función establecerPrimerNombre
22
23 // devuelve el primer nombre
24 string Empleado::obtenerPrimerNombre() const
25 {
26   return primerNombre;
27 } // fin de la función establecerPrimerNombre
28
29 // establece el apellido paterno
30 void Empleado::establecerApellidoPaterno( const string &apellido )
31 {
32   apellidoPaterno = apellido;
33 } // fin de la función establecerApellidoPaterno
34
35 // devuelve el apellido paterno
36 string Empleado::obtenerApellidoPaterno() const
37 {
38   return apellidoPaterno;
39 } // fin de la función obtenerApellidoPaterno
40
41 // establece el número de seguro social
42 void Empleado::establecerNumeroSeguroSocial( const string &nss )
43 {
44   numeroSeguroSocial = nss; // debe validar
45 } // fin de la función establecerNumeroSeguroSocial
46
47 // devuelve el número de seguro social
48 string Empleado::obtenerNumeroSeguroSocial() const
49 {
50   return numeroSeguroSocial;
51 } // fin de la función obtenerNumeroSeguroSocial
52
53 // imprime la información del Empleado (virtual, pero no virtual pura)
54 void Empleado::imprimir() const
55 {
56   cout << obtenerPrimerNombre() << " " << obtenerApellidoPaterno()
57     << "\nnumero de seguro social: " << obtenerNumeroSeguroSocial();
58 } // fin de la función imprimir

```

Fig. 12.10 | Archivo de implementación de la clase Empleado (parte 2 de 2).

La función virtual `imprimir` (líneas 53 a 57) proporciona una *implementación* que se *sobrescribirá* en cada una de las clases derivadas. Sin embargo, cada una de estas funciones utilizará la versión de `imprimir` de la clase abstracta para imprimir información *común para todas las clases* en la jerarquía de `Empleado`.

12.6.2 Creación de la clase derivada concreta `EmpleadoAsalariado`

La clase `EmpleadoAsalariado` (figuras 12.11 y 12.12) se deriva de la clase `Empleado` (línea 9 de la figura 12.11). Las funciones miembro `public` incluyen a un constructor que recibe un primer nombre, un apellido paterno, un número de seguro social y un salario semanal como argumentos (líneas 12 y 13); un destructor virtual (línea 14); una función *establecer* para asignar un nuevo valor no negativo al miembro de datos `salarioSemanal` (línea 16); una función *obtener* para devolver el valor de `salarioSemanal` (línea 17); una función virtual llamada `ingresos` que calcula los ingresos de un `EmpleadoAsalariado` (línea 20) y una función virtual llamada `imprimir` (línea 21) que imprime el tipo del empleado, a saber, "empleado asalariado:" seguido de la información específica del empleado producida por la función `imprimir` de la clase base `Empleado` y la función `obtenerSalarioSemanal` de `EmpleadoAsalariado`.

```

1 // Fig. 12.11: EmpleadoAsalariado.h
2 // Clase EmpleadoAsalariado derivada de Empleado.
3 #ifndef ASALARIADO_H
4 #define ASALARIADO_H
5
6 #include <string> // clase string estándar de C++
7 #include "Empleado.h" // definición de la clase Empleado
8
9 class EmpleadoAsalariado : public Empleado
10 {
11 public:
12     EmpleadoAsalariado( const std::string &, const std::string &,
13                       const std::string &, double = 0.0 );
14     virtual ~EmpleadoAsalariado() {} // destructor virtual
15
16     void establecerSalarioSemanal( double ); // establece el salario semanal
17     double obtenerSalarioSemanal() const; // devuelve el salario semanal
18
19     // la palabra clave virtual indica el intento de sobrescribir
20     virtual double ingresos() const override; // calcula los ingresos
21     virtual void imprimir() const override; // imprime el objeto
22 private:
23     double salarioSemanal; // salario por semana
24 }; // fin de la clase EmpleadoAsalariado
25
26 #endif // ASALARIADO_H

```

Fig. 12.11 Encabezado de la clase `EmpleadoAsalariado`.

Definiciones de las funciones miembro de la clase `EmpleadoAsalariado`

La figura 12.12 contiene las definiciones de las funciones miembro para `EmpleadoAsalariado`. El constructor de la clase pasa el primer nombre, apellido paterno y número de seguro social al constructor de `Empleado` (línea 11) para inicializar los miembros de datos `private` que se heredan de la clase base, pero que no se pueden utilizar directamente en la clase derivada. La función `ingresos` (líneas 33 a 36) sobrescribe la función virtual pura `ingresos` en `Empleado` para proporcionar una implementación

concreta que devuelva el salario semanal del `EmpleadoAsalariado`. Si no definiéramos `ingresos`, la clase `EmpleadoAsalariado` sería una clase *abstracta* y cualquier intento de instanciar un objeto de la clase produciría un error de compilación. En el encabezado de la clase `EmpleadoAsalariado` declaramos las funciones miembro `ingresos` e `imprimir` como `virtual` (líneas 20 y 21 de la figura 12.11); en realidad, colocar la palabra clave `virtual` antes de estas funciones miembro es *redundante*. Las definimos como `virtual` en la clase base `Empleado`, por lo que siguen siendo funciones `virtual` a través de la jerarquía de clases. Declarar dichas funciones explícitamente como `virtual` en cada nivel de la jerarquía puede promover la claridad del programa. Si no declaramos `ingresos` como `virtual` pura, indicamos nuestra intención de proporcionar una implementación en esta clase concreta.

```

1 // Fig. 12.12: EmpleadoAsalariado.cpp
2 // Definiciones de las funciones miembro de la clase EmpleadoAsalariado.
3 #include <iostream>
4 #include <stdexcept>
5 #include "EmpleadoAsalariado.h" // definición de la clase EmpleadoAsalariado
6 using namespace std;
7
8 // constructor
9 EmpleadoAsalariado::EmpleadoAsalariado( const string &nombre,
10     const string &apellido, const string &nss, double salario )
11     : Empleado( nombre, apellido, nss )
12 {
13     establecerSalarioSemanal( salario );
14 } // fin del constructor de EmpleadoAsalariado
15
16 // establece el salario
17 void EmpleadoAsalariado::establecerSalarioSemanal( double salario )
18 {
19     if ( salario >= 0.0 )
20         salarioSemanal = salario;
21     else
22         throw invalid_argument( "El salario semanal debe ser >= 0.0" );
23 } // fin de la función establecerSalarioSemanal
24
25 // devuelve el salario
26 double EmpleadoAsalariado::obtenerSalarioSemanal() const
27 {
28     return salarioSemanal;
29 } // fin de la función obtenerSalarioSemanal
30
31 // calcula los ingresos;
32 // sobrescribe a la función virtual pura ingresos en Empleado
33 double EmpleadoAsalariado::ingresos() const
34 {
35     return obtenerSalarioSemanal();
36 } // fin de la función ingresos
37
38 // imprime la información del EmpleadoAsalariado
39 void EmpleadoAsalariado::imprimir() const
40 {

```

Fig. 12.12 | Archivo de implementación de la clase `EmpleadoAsalariado` (parte 1 de 2).


```

41     cout << "Empleado asalariado: ";
42     Empleado::imprimir(); // reutiliza la función imprimir de la clase base abstracta
43     cout << "\nsalario semanal: " << obtenerSalarioSemanal();
44 } // fin de la función imprimir

```

Fig. 12.12 | Archivo de implementación de la clase EmpleadoAsalariado (parte 2 de 2).

La función `imprimir` de la clase `EmpleadoAsalariado` (líneas 39 a 44 de la figura 12.12) sobrescribe a la función `imprimir` de `Empleado`. Si la clase `EmpleadoAsalariado` no sobrescribiera a `imprimir`, `EmpleadoAsalariado` heredaría la versión de `imprimir` correspondiente a `Empleado`. En ese caso, la función `imprimir` de `EmpleadoAsalariado` simplemente devolvería el nombre completo del empleado y el número de seguro social, lo cual no representa en forma adecuada a un `EmpleadoAsalariado`. Para imprimir la información completa de un `EmpleadoAsalariado`, la función `imprimir` de la clase derivada imprime el texto "Empleado asalariado: " seguido de la información específica de la clase base `Empleado` (es decir, primer nombre, apellido paterno y número de seguro social) que se imprime al *invocar la función imprimir de la clase base*, usando el operador de resolución de ámbito (línea 42); éste es un buen ejemplo de reutilización de código. Sin el operador de resolución de ámbito, la llamada a `imprimir` provocaría *recursividad infinita*. Los resultados producidos por la función `imprimir` de `EmpleadoAsalariado` también contienen el salario semanal del empleado que se obtiene al invocar la función `obtenerSalarioSemanal` de la clase.

12.6.3 Creación de la clase derivada concreta EmpleadoPorComision

La clase `EmpleadoPorComision` (figuras 12.13 y 12.14) se deriva de la clase `Empleado` (figura 12.13, línea 9). Las implementaciones de las funciones miembro (figura 12.14) incluyen a un constructor (líneas 9 a 15) que recibe un primer nombre, un apellido paterno, un número de seguro social, un monto de ventas y una tarifa de comisión; funciones *establecer* (líneas 18 a 24 y 33 a 39) para asignar nuevos valores a los miembros de datos `tarifaComision` y `ventasBrutas`, respectivamente; funciones *obtener* (líneas 27 a 30 y 42 a 45) que obtienen sus valores; la función *ingresos* (líneas 48 a 51) para calcular los ingresos de un `EmpleadoPorComision` y la función `imprimir` (líneas 54 a 60), que imprime el tipo del empleado, a saber, "Empleado por comision: ", y la información específica del empleado. El constructor pasa el primer nombre, apellido paterno y número de seguro social al constructor de `Empleado` (línea 11) para inicializar los miembros de datos `private` de `Empleado`. La función `imprimir` llama a la función `imprimir` de la clase base (línea 57) para mostrar la información específica de `Empleado`.

```

1 // Fig. 12.13: EmpleadoPorComision.h
2 // Clase EmpleadoPorComision derivada de Empleado.
3 #ifndef COMISION_H
4 #define COMISION_H
5
6 #include <string> // clase string estándar de C++
7 #include "Empleado.h" // definición de la clase Empleado
8
9 class EmpleadoPorComision : public Empleado
10 {
11 public:
12     EmpleadoPorComision( const std::string &, const std::string &,
13         const std::string &, double = 0.0, double = 0.0 );

```

Fig. 12.13 | Encabezado de la clase EmpleadoPorComision (parte 1 de 2).

```

14 virtual ~EmpleadoPorComision() { } // destructor virtual
15
16 void establecerTarifaComision( double ); // establece la tarifa de comisión
17 double obtenerTarifaComision() const; // devuelve la tarifa de comisión
18
19 void establecerVentasBrutas( double ); // establece el monto de ventas brutas
20 double obtenerVentasBrutas() const; // devuelve el monto de ventas brutas
21
22 // la palabra clave virtual indica la intención de sobrescribir
23 virtual double ingresos() const override; // calcula los ingresos
24 virtual void imprimir() const override; // imprime el objeto
25 private:
26 double ventasBrutas; // ventas brutas semanales
27 double tarifaComision; // porcentaje de comisión
28 }; // fin de la clase EmpleadoPorComision
29
30 #endif // COMISION_H

```

Fig. 12.13 | Encabezado de la clase `EmpleadoPorComision` (parte 2 de 2).

```

1 // Fig. 12.14: EmpleadoPorComision.cpp
2 // Definiciones de las funciones miembro de la clase EmpleadoPorComision.
3 #include <iostream>
4 #include <stdexcept>
5 #include "EmpleadoPorComision.h" // definición de la clase EmpleadoPorComision
6 using namespace std;
7
8 // constructor
9 EmpleadoPorComision::EmpleadoPorComision( const string &nombre,
10 const string &apellido, const string &nss, double ventas, double tarifa )
11 : Empleado( nombre, apellido, nss )
12 {
13     establecerVentasBrutas( ventas );
14     establecerTarifaComision( tarifa );
15 } // fin del constructor de EmpleadoPorComision
16
17 // establece el monto de ventas brutas
18 void EmpleadoPorComision::establecerVentasBrutas( double ventas )
19 {
20     if ( ventas >= 0.0 )
21         ventasBrutas = ventas;
22     else
23         throw invalid_argument( "Las ventas brutas deben ser >= 0.0" );
24 } // fin de la función establecerVentasBrutas
25
26 // devuelve el monto de ventas brutas
27 double EmpleadoPorComision::obtenerVentasBrutas() const
28 {
29     return ventasBrutas;
30 } // fin de la función obtenerVentasBrutas
31

```

Fig. 12.14 | Archivo de implementación de la clase `EmpleadoPorComision` (parte 1 de 2).


```

32 // establece la tarifa de comisión
33 void EmpleadoPorComision::establecerTarifaComision( double tarifa )
34 {
35     if ( tarifa > 0.0 && tarifa < 1.0 )
36         tarifaComision = tarifa;
37     else
38         throw invalid_argument( "La tarifa de comision debe ser > 0.0 y < 1.0" );
39 } // fin de la función establecerTarifaComision
40
41 // devuelve la tarifa de comisión
42 double EmpleadoPorComision::obtenerTarifaComision() const
43 {
44     return tarifaComision;
45 } // fin de la función obtenerTarifaComision
46
47 // calcula los ingresos: sobrescribe la función virtual pura ingresos en Empleado
48 double EmpleadoPorComision::ingresos() const
49 {
50     return obtenerTarifaComision() * obtenerVentasBrutas();
51 } // fin de la función ingresos
52
53 // imprime la información del EmpleadoPorComision
54 void EmpleadoPorComision::imprimir() const
55 {
56     cout << "Empleado por comision: ";
57     Empleado::imprimir(); // reutilización de código
58     cout << "\nventas brutas: " << obtenerVentasBrutas()
59         << "; tarifa de comision: " << obtenerTarifaComision();
60 } // fin de la función imprimir

```

Fig. 12.14 | Archivo de implementación de la clase EmpleadoPorComision (parte 2 de 2).

12.6.4 Creación de la clase derivada concreta indirecta EmpleadoBaseMasComision

La clase EmpleadoBaseMasComision (figuras 12.15 y 12.16) hereda directamente de la clase EmpleadoPorComision (línea 9 de la figura 12.15), y por lo tanto es una clase derivada *indirecta* de la clase Empleado. Las implementaciones de las funciones miembro de la clase EmpleadoBaseMasComision incluyen a un constructor (líneas 9 a 15 de la figura 12.16) que recibe como argumentos un primer nombre, un apellido paterno, un número de seguro social, un monto de ventas, una tarifa de comisión y un salario base. Después pasa el primer nombre, apellido paterno, número de seguro social, monto de ventas y tarifa de comisión al constructor de EmpleadoPorComision (línea 12) para inicializar los miembros heredados. EmpleadoBaseMasComision también contiene una función *establecer* (líneas 18 a 24) para asignar un nuevo valor al miembro de datos salarioBase y una función *obtener* (líneas 27 a 30) para devolver el valor de salarioBase. La función ingresos (líneas 34 a 37) calcula los ingresos de un EmpleadoBaseMasComision. En la línea 36 de la función ingresos se llama a la función ingresos de la clase base EmpleadoPorComision para calcular la porción basada en comisiones de los ingresos del empleado. Éste es otro buen ejemplo de reutilización de código. La función imprimir de EmpleadoBaseMasComision (líneas 40 a 45) imprime el texto "con salario base", seguido de los resultados de la función imprimir de la clase base EmpleadoPorComision (otro ejemplo de reutilización de código), y después el salario base. La salida resultante empieza con el texto "con salario base empleado por comision:" seguido del resto de la información de Empleado-BaseMasComision. Recuerde que la función imprimir de EmpleadoPorComision muestra el primer nombre,

apellido paterno y número de seguro social del empleado al invocar la función `imprimir` de su clase base (es decir, `Empleado`); otro ejemplo más de reutilización de código. La función `imprimir` de `EmpleadoBaseMasComision` inicia una cadena de funciones que abarca *los tres niveles* de la jerarquía de `Empleado`.

```

1 // Fig. 12.15: EmpleadoBaseMasComision.h
2 // Clase EmpleadoBaseMasComision derivada de EmpleadoPorComision.
3 #ifndef BASEMAS_H
4 #define BASEMAS_H
5
6 #include <string> // clase string estándar de C++
7 #include "EmpleadoPorComision.h" // definición de la clase EmpleadoPorComision
8
9 class EmpleadoBaseMasComision : public EmpleadoPorComision
10 {
11 public:
12     EmpleadoBaseMasComision( const std::string &, const std::string &,
13                             const std::string &, double = 0.0, double = 0.0, double = 0.0 );
14     virtual ~EmpleadoPorComision() { } // destructor virtual
15
16     void establecerSalarioBase( double ); // establece el salario base
17     double obtenerSalarioBase() const; // devuelve el salario base
18
19     // la palabra clave virtual indica el intento de sobrescribir
20     virtual double ingresos() const override; // calcula los ingresos
21     virtual void imprimir() const override; // imprime el objeto
22 private:
23     double salarioBase; // salario base por semana
24 }; // fin de la clase EmpleadoBaseMasComision
25
26 #endif // BASEMAS_H

```

Fig. 12.15 | Encabezado de `EmpleadoBaseMasComision`.

```

1 // Fig. 12.16: EmpleadoBaseMasComision.cpp
2 // Definiciones de las funciones miembro de EmpleadoBaseMasComision.
3 #include <iostream>
4 #include <stdexcept>
5 #include "EmpleadoBaseMasComision.h"
6 using namespace std;
7
8 // constructor
9 EmpleadoBaseMasComision::EmpleadoBaseMasComision(
10     const string &nombre, const string &apellido, const string &nss,
11     double ventas, double tarifa, double salario )
12     : EmpleadoPorComision( nombre, apellido, nss, ventas, tarifa )
13 {
14     establecerSalarioBase( salario ); // valida y almacena el salario base
15 } // fin del constructor de EmpleadoBaseMasComision
16

```

Fig. 12.16 | Archivo de implementación de la clase `EmpleadoBaseMasComision` (parte 1 de 2).


```

17 // establece el salario base
18 void EmpleadoBaseMasComision::establecerSalarioBase( double salario )
19 {
20     if ( salario >= 0.0 )
21         salarioBase = salario;
22     else
23         throw invalid_argument( "El salario debe ser >= 0.0" );
24 } // fin de la función establecerSalarioBase
25
26 // devuelve el salario base
27 double EmpleadoBaseMasComision::obtenerSalarioBase() const
28 {
29     return salarioBase;
30 } // fin de la función obtenerSalarioBase
31
32 // calcula los ingresos;
33 // sobrescribe la función virtual ingresos en EmpleadoPorComision
34 double EmpleadoBaseMasComision::ingresos() const
35 {
36     return obtenerSalarioBase() + EmpleadoPorComision::ingresos();
37 } // fin de la función ingresos
38
39 // imprime la información del EmpleadoBaseMasComision
40 void EmpleadoBaseMasComision::imprimir() const
41 {
42     cout << "con salario base ";
43     EmpleadoPorComision::imprimir(); // reutilización de código
44     cout << "; salario base: " << obtenerSalarioBase();
45 } // fin de la función imprimir

```

Fig. 12.16 | Archivo de implementación de la clase EmpleadoBaseMasComision (parte 2 de 2).

12.6.5 Demostración del procesamiento polimórfico

Para evaluar nuestra jerarquía de Empleado, el programa de la figura 12.17 crea un objeto de cada una de las tres clases concretas EmpleadoAsalariado, EmpleadoPorComision y EmpleadoBaseMasComision. El programa manipula estos objetos, primero con la *vinculación estática* y después en forma *polimórfica*, usando un vector de apuntadores Empleado. En las líneas 22 a 27 se crean objetos de cada una de las tres clases derivadas concretas de Empleado. En las líneas 32 a 38 se imprime la información y los ingresos de cada Empleado. La invocación a cada función miembro en las líneas 32 a 37 es un ejemplo de *vinculación estática*; en tiempo de compilación, puesto que estamos usando *manejadores de nombres* (y no *apuntadores* o *referencias* que podrían establecerse en tiempo de ejecución), el compilador puede identificar el tipo de cada objeto para determinar cuáles funciones de imprimir e ingresos se van a llamar.

```

1 // Fig. 12.17: fig12_17.cpp
2 // Procesamiento de objetos de clases derivadas de Empleado en forma
3 // individual y polimórfica, mediante el uso de la vinculación dinámica.
4 #include <iostream>
5 #include <iomanip>
6 #include <vector>

```

Fig. 12.17 | Programa controlador de la jerarquía de clases de Empleado (parte 1 de 4).

```

7  #include "Empleado.h"
8  #include "EmpleadoAsalariado.h"
9  #include "EmpleadoPorComision.h"
10 #include "EmpleadoBaseMasComision.h"
11 using namespace std;
12
13 void virtualViaApuntador( const Empleado * const ); // prototipo
14 void virtualViaReferencia( const Empleado & ); // prototipo
15
16 int main()
17 {
18     // establece el formato de salida de punto flotante
19     cout << fixed << setprecision( 2 );
20
21     // crea objetos de las clases derivadas
22     EmpleadoAsalariado empleadoAsalariado(
23         "John", "Smith", "111-11-1111", 800 );
24     EmpleadoPorComision empleadoPorComision(
25         "Sue", "Jones", "333-33-3333", 10000, .06 );
26     EmpleadoBaseMasComision empleadoBaseMasComision(
27         "Bob", "Lewis", "444-44-4444", 5000, .04, 300 );
28
29     cout << "Empleados procesados en forma individual, usando vinculación
30         estática:\n\n";
31
32     // imprime la información de cada empleado y sus ingresos, usando vinculación
33     // estática
34     empleadoAsalariado.imprimir();
35     cout << "\nbtuvo $" << empleadoAsalariado.ingresos() << "\n\n";
36     empleadoPorComision.imprimir();
37     cout << "\nbtuvo $" << empleadoPorComision.ingresos() << "\n\n";
38     empleadoBaseMasComision.imprimir();
39     cout << "\nbtuvo $" << empleadoBaseMasComision.ingresos()
40         << "\n\n";
41
42     // crea un vector de tres apuntadores de la clase base
43     vector< Empleado * > empleados( 3 );
44
45     // inicializa el vector con apuntadores a objetos Empleado
46     empleados[ 0 ] = &empleadoAsalariado;
47     empleados[ 1 ] = &empleadoPorComision;
48     empleados[ 2 ] = &empleadoBaseMasComision;
49
50     cout << "Empleados procesados en forma polimorfica mediante vinculacion
51         dinámica:\n\n";
52
53     // llama a virtualViaApuntador para imprimir la información de cada Empleado
54     // y a ingresos mediante el uso de la vinculación dinámica
55     cout << "Llamadas a funciones virtuales realizadas desde apuntadores de la
56         clase base:\n\n";
57
58     for ( const Empleado *empleadoPtr : empleados )
59         virtualViaApuntador( empleadoPtr );
60
61     // llama a virtualViaReferencia para imprimir la información de cada Empleado
62     // y a ingresos mediante el uso de vinculación dinámica
63     cout << "Llamadas a funciones virtuales realizadas desde referencias de la
64         clase base:\n\n";

```

Fig. 12.17 | Programa controlador de la jerarquía de clases de Empleado (parte 2 de 4).


```

60
61     for ( const Empleado *empleadoPtr : empleados )
62         virtualViaReferencia( *empleadoPtr ); // observe la desreferencia
63 } // fin de main
64
65 // llama a las funciones virtuales imprimir e ingresos de Empleado desde un
66 // apuntador de la clase base mediante la vinculación dinámica
67 void virtualViaApuntador( const Empleado * const claseBasePtr )
68 {
69     claseBasePtr->imprimir();
70     cout << "\n obtuvo $" << claseBasePtr->ingresos() << "\n\n";
71 } // fin de la función virtualViaApuntador
72
73 // llama a las funciones virtuales imprimir e ingresos de Empleado desde una
74 // referencia de la clase base mediante la vinculación dinámica
75 void virtualViaReferencia( const Empleado &claseBaseRef )
76 {
77     claseBaseRef.imprimir();
78     cout << "\n obtuvo $" << claseBaseRef.ingresos() << "\n\n";
79 } // fin de la función virtualViaReferencia

```

Empleados procesados en forma individual, usando vinculacion estatica:

```

empleado asalariado: John Smith
numero de seguro social: 111-11-1111
salario semanal: 800.00
obtuvo $800.00

empleado por comision: Sue Jones
numero de seguro social: 333-33-3333
ventas brutas: 10000.00; tarifa de comision: 0.06
obtuvo $600.00

con salario base empleado por comision: Bob Lewis
numero de seguro social: 444-44-4444
ventas brutas: 5000.00; tarifa de comision: 0.04; salario base: 300.00
obtuvo $500.00

```

Empleados procesados en forma polimorfica mediante vinculacion dinamica:

Llamadas a funciones virtuales realizadas desde apuntadores de la clase base:

```

empleado asalariado: John Smith
numero de seguro social: 111-11-1111
salario semanal: 800.00
obtuvo $800.000

empleado por comision: Sue Jones
numero de seguro social: 333-33-3333
ventas brutas: 10000.00; tarifa de comision: 0.06
obtuvo $600.00

con salario base empleado por comision: Bob Lewis
numero de seguro social: 444-44-4444
ventas brutas: 5000.00; tarifa de comision: 0.04; salario base: 300.00
obtuvo $500.00

```

Fig. 12.17 Programa controlador de la jerarquia de clases de Empleado (parte 3 de 4).

Llamadas a funciones virtuales realizadas desde referencias de la clase base:

```

empleado asalariado: John Smith
numero de seguro social: 111-11-1111
salario semanal: 800.00
obtuvo $800.00

empleado por comision: Sue Jones
numero de seguro social: 333-33-3333
ventas brutas: 10000.00; tarifa de comision: 0.06
obtuvo $600.00

con salario base empleado por comision: Bob Lewis
numero de seguro social: 444-44-4444
ventas brutas: 5000.00; tarifa de comision: 0.04; salario base: 300.00
obtuvo $500.00

```

Fig. 12.17 | Programa controlador de la jerarquía de clases de Empleado (parte 4 de 4).

En la línea 41 se crea el vector `empleados`, que contiene tres apuntadores `Empleado`. En la línea 44 se orienta `empleados[0]` al objeto `EmpleadoAsalariado`. En la línea 45 se orienta `empleados[1]` al objeto `EmpleadoPorComision`. En la línea 46 se orienta `empleados[2]` al objeto `EmpleadoBaseMasComision`. El compilador permite estas asignaciones, ya que un `EmpleadoAsalariado` *es un* `Empleado`, un `EmpleadoPorComision` *es un* `Empleado` y un `EmpleadoBaseMasComision` *es un* `Empleado`. Por lo tanto, podemos asignar las direcciones de los objetos `EmpleadoAsalariado`, `EmpleadoPorComision` y `EmpleadoBaseMasComision` a los apuntadores de la clase base `Empleado` (aun cuando `Empleado` sea una clase abstracta).

El ciclo en las líneas 54 y 55 recorre el vector `empleados` e invoca a la función virtual `ViaApuntador` (líneas 67 a 71) para cada elemento en `empleados`. La función virtual `ViaApuntador` recibe en el parámetro `claseBasePtr` la dirección almacenada en un elemento de `empleados`. Cada llamada a `virtualViaApuntador` utiliza a `claseBasePtr` para invocar a las funciones virtual `imprimir` (línea 69) e `ingresos` (línea 70). La función virtual `ViaApuntador` *no contiene ninguna* información de los tipos `EmpleadoAsalariado`, `EmpleadoPorComision` o `EmpleadoBaseMasComision`. La función *sólo* sabe acerca del tipo de su clase base `Empleado`. Por lo tanto, el compilador *no puede saber* cuáles funciones de la clase concreta debe llamar a través de `claseBasePtr`. Aún en tiempo de ejecución, cada invocación a una función virtual llama *correctamente* a la función en el objeto al que apunta `claseBasePtr` en ese momento. La salida ilustra que *sin duda se invocan las funciones apropiadas para cada clase*, y que se muestra la información apropiada de cada objeto. Por ejemplo, el salario semanal se muestra para el `EmpleadoAsalariado`, y las ventas brutas se muestran para `EmpleadoPorComision` y `EmpleadoBaseMasComision`. Además, al obtener los ingresos de cada `Empleado` en forma polimórfica en la línea 70 se producen los mismos resultados que obtener los ingresos de esos empleados mediante la *vinculación estática* en las líneas 33, 35 y 37. Todas las llamadas a las funciones virtual `imprimir` e `ingresos` se resuelven en *tiempo de ejecución* con la *vinculación dinámica*.

Por último, el ciclo en las líneas 61 y 62 recorre a `empleados` e invoca la función virtual `ViaReferencia` (líneas 75 a 79) para cada elemento en el vector. La función virtual `ViaReferencia` recibe en su parámetro `claseBaseRef` (de tipo `const Empleado &`) una *referencia* al objeto, que se obtiene al *desreferenciar el apuntador* almacenado en cada elemento de `empleados` (línea 62). Cada llamada a `virtualViaReferencia` invoca a las funciones virtual `imprimir` (línea 77) e `ingresos` (línea 78) a través de la referencia `claseBaseRef` para demostrar que *el procesamiento polimórfico ocurre también con las referencias de la clase base*. Cada invocación a una función virtual llama a la función en el objeto

al que `claseBaseRef` hace referencia en tiempo de ejecución. Éste es otro ejemplo de *vinculación dinámica*. Los resultados producidos usando referencias de la clase base son idénticos a los resultados que se producen usando apuntadores de la clase base.

12.7 (Opcional) Polimorfismo, funciones virtuales y vinculación dinámica “detrás de las cámaras”

C++ facilita la programación del polimorfismo. Evidentemente, es posible programar para el polimorfismo en los lenguajes no orientados a objetos como C, pero para ello se requieren manipulaciones de apuntadores complejas y potencialmente peligrosas. Esta sección habla acerca de cómo C++ puede implementar el polimorfismo, las funciones virtual y la vinculación dinámica de manera interna. Esto proporcionará al lector una sólida comprensión de la forma en que realmente funcionan estas herramientas. Lo que es más importante, le ayudará a apreciar la *sobrecarga* del polimorfismo; en términos de *consumo de memoria* y *tiempo de procesador* adicionales. Esto le ayudará a determinar cuándo usar el polimorfismo y cuándo evitarlo. Las clases de la Biblioteca estándar de C++ como `array` y `vector` se implementan *sin* polimorfismo ni funciones virtual para evitar la sobrecarga asociada en tiempo de ejecución y lograr un rendimiento óptimo.

Primero explicaremos las estructuras de datos que el compilador genera en *tiempo de compilación* para dar soporte al polimorfismo en tiempo de ejecución. Veremos que el polimorfismo se lleva a cabo a través de tres niveles de apuntadores (es decir, *triple indirección*). Después mostraremos cómo un programa en ejecución utiliza estas estructuras de datos para ejecutar funciones virtual y lograr la *vinculación dinámica* asociada con el polimorfismo. Nuestra discusión explica una *posible* implementación; esto no es un requerimiento del lenguaje.

Cuando C++ compila una clase que tiene una o más funciones virtual, genera una **tabla de funciones virtuales** (*vtable*) para esa clase. La *vtable* contiene apuntadores a las funciones virtual de la clase. Así como el nombre de un arreglo integrado contiene la dirección en memoria del primer elemento del arreglo, un **apuntador a una función** contiene la dirección inicial en memoria del código que realiza la tarea de la función. Un programa en ejecución utiliza la *vtable* para seleccionar la implementación de la función apropiada cada vez que se llama a una función virtual de esa clase. La columna de más a la izquierda de la figura 12.18 ilustra las *vtables* para las clases `Empleado`, `EmpleadoAsalariado`, `EmpleadoPorComision` y `EmpleadoBaseMasComision`.

Vtable de la clase `Empleado`

En la *vtable* para la clase `Empleado`, el apuntador a la primera función se establece en 0 (es decir, `nullptr`), ya que la función `ingresos` es una función virtual *pura*, y por lo tanto *carece de una implementación*. El apuntador a la segunda función apunta a la función `imprimir`, la cual muestra el nombre completo y número de seguro social del empleado. [Nota: hemos abreviado la salida de cada función `imprimir` en esta figura para conservar espacio]. Cualquier clase que tenga uno o más apuntadores nulos en su *vtable* es una clase abstracta. Las clases sin apuntadores nulos en su *vtable* (como `EmpleadoAsalariado`, `EmpleadoPorComision` y `EmpleadoBaseMasComision`) son clases *concretas*.

Vtable de la clase `EmpleadoAsalariado`

La clase `EmpleadoAsalariado` sobrescribe a la función `ingresos` para regresar el salario semanal del empleado, de manera que el apuntador a función apunta a la función `ingresos` de la clase `EmpleadoAsalariado`. Esta clase también sobrescribe a `imprimir`, por lo que el apuntador a la función correspondiente apunta a la función miembro de `EmpleadoAsalariado` que imprime el texto “empleado asalariado: ” seguido del nombre del empleado, su número de seguro social y su salario semanal.